

LED Lab 1

You are now going to work your way through the first five projects. These projects all use LED lights in various ways. You will learn about controlling outputs from the Arduino as well as simple inputs such as button presses. On the hardware side, you will learn about LEDs, resistors, etc. Along the way you will be able to pick up the concepts of programming in the Arduino language.

Lab 1 – Singular LED Blink

For the first project, you are going to be connecting an LED to one of the digital pins. You will also learn exactly how the hardware and the software for this project works, learning a bit about electronics and coding in the Arduino language which is a variant of C all in once.

Connecting Everything

First, make sure your Arduino is ALWAYS powered off by unplugging it from the USB Cable. Now, take your breadboard, LED, resistor, and wires and connect everything as show in the image below.

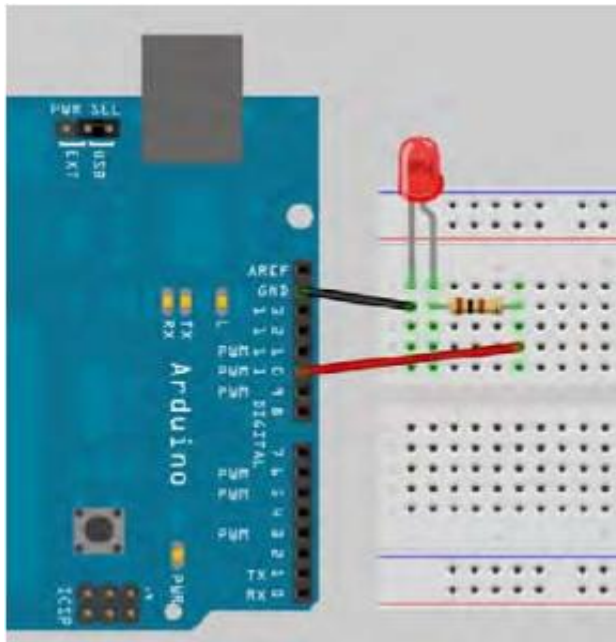


Figure 1 – The Circuit for Lab 1: Singular LED Blink

It doesn't matter if you use different colored wires or use different holes on the breadboard as long as the components and wires are connected in the same order as in the picture. Be careful when inserting components into the breadboard. If your breadboard is brand

new, the grips in the holes will be stiff, Failure to insert components carefully could result in damage.

Make sure that your LED is connected correctly with the longer leg connected to Digital Pin 10. The long leg is the anode of the LED and must ALWAYS go to the +5v supply (in the case, coming out of the Digital Pin 10); the short legs the cathode and must go to the GND (ground). When you are sure that everything is connected correctly, power up your Arduino and connect the USB cable.

[Enter the Code](#)

Open your Arduino IDE and type in the code below.

// Mini Lab: Singular LED Blink

```
Int LedPin =10;  
void setup () {  
    pinMode(LedPin, OUTPUT);  
}  
void loop() {  
    digitalWrite(LedPin, HIGH);  
    delay(1000);  
    digitalWrite(LedPin, LOW);  
    delay(1000);  
}
```

Press the Verify/Compile button at the top of the IDE to make sure there are no errors in your code. If this is successful, click the upload button to upload the code to your Arduino. If you have done everything right, you should now see the colored LED on the breadboard flashing on and off every second. Let's look at the code and the hardware to find out how they both work.

[Mini Lab: Singular LED Blink \(Code Overview\)](#)

The first line of code for this project is:

```
// Mini Lab: Singular LED Blink
```

This is just a comment in your code. We notice it's a comment because it starts with `/**` and any text that begins this way will be ignored by the compiler. Comments are essential in your code; they help you understand how your code works. As your projects get more complex and your code expands into hundreds of thousands of lines, comments will be vital in making it easy for users to understand how each section functions. You may come up with an amazing piece of code, but you can't count on remembering how it works when you revisit it several days, weeks, or months later. Comments, however, will remind you of its functionality. Also, if your code is meant to be seen by other people, comments will help that person understand what is going on in your code. The whole ethos of the Arduino, and indeed the whole Open Source Community, is to share code and schematics. I hope that when you start making your own cool stuff with the Arduino you will be willing to share it with the world, too.

There is another format for making comments; it is called a block statement, which is shown by `/*` and `*/`, it is used as shown below:

**`/* Copy all of this text inside the slash and
the asterisks in the IDE compiler
and notice how it will be ignored by the compiler
because this is another form of commenting*/`**

Note: The IDE will automatically turn the color of any commented text to grey. The next line of the program is

`int LedPin = 10`

and this is known as a variable. A variable is a place to store data. In this case we are setting up a variable of type int or integer. An integer is a number within the range of -32,768 to 32,767. Next, you have assigned that integer the name of ledPin and have given it a value of 10. (You didn't have to call it ledPin, you could have called it anything you wanted to. But you want your variable name to be descriptive, so you call it ledPin to show that this variable sets which pin on the Arduino you are going to use to connect your LED.) In this case, you are using Digital Pin 10. At the end of this statement is a semicolon. This symbol tells the compiler that this statement is now complete.

Although you can call your variables anything, every variable name in C must start with a letter; the rest of the name can consist of letters, numbers, and underscore characters. Note that C recognizes upper and lower case characters as being different. Finally, you cannot use any of C's keywords like `main`, `while`, `switch` etc as variable names. Keywords are constants, variables, and function names that are defined as part of the Arduino language. To help you avoid naming a variable after a keyword, all keywords within the sketch will appear in red.

Imagine a variable as a small box where you can keep things. So in this sketch, you have set up an area in memory to store a number of type integer and have stored in that area the number 10.

Finally, a variable is called a variable because you can change it. Later, you will carry out mathematical calculations on variables to make your program do more advanced things. Next is your `setup()` function:

```
void setup() {  
  
    pinMode(LedPin, OUTPUT);  
  
}
```

An Arduino sketch must have a `setup()` and `loop()` function, otherwise it will not work. The `setup()` function runs once and once only at the start of the program and is where you will issue general instructions to prepare the program before the main loop runs, such as setting up pin modes, setting serial baud rates, etc. Basically, a function is a bunch of code assembled into one convenient block. For example, if you created your own function to carry out a series of complicated mathematics that had many lines of code, you could run that code as many times as you liked simply by calling the function name instead of writing out the code again each time. You will go into functions in more detail later when you start to create your own. In the case of this program, however, the `setup()` function only has one statement to carry out. The function starts with

```
void setup()
```

This tells the compiler that your function is called `setup`, that it returns no data (`void`), and that you pass no parameters to it (empty parenthesis). If your function returned an integer value and you also had integer values to pass to it (e.g. for the function to process), it would look something like this:

```
int myFunc(int x, int y)
```

Here you have created a function (or a block of code) called myFunc. This function has been passed two integers called x and y. Once the function has finished, it will then return an integer value to the point after where your function was called in the program (hence int before the function name).

All of the code within the function is contained within the curly braces. A { symbol starts the block of code and a } symbol ends the block. Anything in between those two symbols is code that belongs to the function. (I will go into greater detail about functions later, so don't worry about them for now.) In this program, you have two functions; the first function is called setup and its purpose is to setup anything necessary for your program to work before the main program loop runs:

```
void setup() {  
  
    pinMode(LedPin, OUTPUT);  
  
}
```

Your setup function only has one statement and that is pinMode, which telling the Arduino that you want to set the mode of one of your pins to be Output mode, rather than Input. Within the parenthesis, you put the pin number and the mode (OUTPUT or INPUT). Your pin number is ledPin, which has been previously set to the value 10. Therefore, this statement is simply telling the Arduino that Digital Pin 10 is to be set to OUTPUT mode. As the setup() function runs only once, you now move onto the main function loop:

```
void loop() {  
  
    digitalWrite(LedPin, HIGH);  
  
    delay(1000);  
  
    digitalWrite(LedPin, LOW);  
  
    delay(1000);  
  
}
```

The loop() function is the main program function and runs continuously as long as the Arduino is turned on. Every statement within the loop() function (within the curly braces) is carried out, one by one, step by step, until the bottom of the function is reached, then the loop starts again at the top of the function, and so on forever or until you turn the Arduino off or press the Reset switch. In this project, you want the LED to turn on, stay on for one second, turn off and remain off for one second, and then repeat. The commands to tell the Arduino to do this

are contained within the loop() function because you wish them to repeat over and over. The first statement is

digitalWrite(LedPin, HIGH);

This writes a HIGH or a LOW value to the pin within the statement (in this case ledPin, which is Digital Pin 10). When you set a pin to HIGH, you are sending out 5 volts to that pin. When you set it to LOW, the pin becomes 0 volts, or ground. This statement, therefore, sends out 5v to pin 10 and turns the LED on. After that is

Delay(1000);

and this statement simply tells the Arduino to wait for 1000 milliseconds (there are 1000 milliseconds in a second) before carrying out the next statement of

digitalWrite(LedPin, LOW);

which will turn off the power going to Digital Pin 10 and therefore turn the LED off. Then there is another delay statement for another 1000 milliseconds and then the function ends. However, as this is your main loop() function, the function will start again at the beginning.

By following the program structure step by step again, you can see that it is very simple:

// Project 1 - LED Flasher

int LedPin = 10;

void setup() {

pinMode(LedPin, OUTPUT);

}

void loop() {

digitalWrite(LedPin, HIGH);

delay(1000);

digitalWrite(LedPin, LOW);

delay(1000);

}

You start off by assigning a variable called ledPin, giving that variable a value of 10. Then you move on to the setup() function where you set the mode for Digital Pin 10 as an output. In the main program loop, you set Digital Pin 10 to high, sending out 5v. Then you wait for a second

and then turn off the 5v to Digital Pin 10, before waiting another second. The loop then starts again at the beginning: the LED will turn on and off continuously for as long as the Arduino has power. Now that you know this, you can modify the code to turn the LED on for a different period of time and turn it off for a different time period. For example, if you wanted the LED to stay on for 2 seconds, then go off for half a second, you could do the following:

```
void loop() {  
  
    digitalWrite(LedPin, HIGH);  
  
    delay(2000);  
  
    digitalWrite(LedPin, LOW);  
  
    delay(500);  
  
}
```

If you would like the LED to stay off for 5 seconds and then flash briefly (250ms), like the LED indicator on a car alarm, you could do this:

```
void loop() {  
  
    digitalWrite(LedPin, HIGH);  
  
    delay(250);  
  
    digitalWrite(LedPin, LOW);  
  
    delay(5000);  
  
}
```

To make the LED flash on and off very fast, try this:

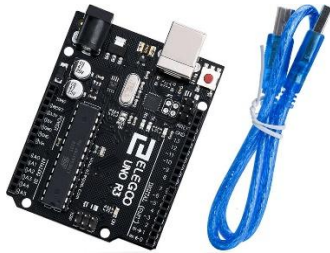
```
void loop() {  
  
    digitalWrite(LedPin, HIGH);  
  
    delay(50);  
  
    digitalWrite(LedPin, LOW);  
  
    delay(50); }
```

By varying the on and off times of the LED you create any effect you want (well, within the bounds of a single LED going on and off). Before you move onto something a little more exciting, let's take a look at the hardware and see how it works.

Mini Lab: Singular LED Blink (Hardware Overview)

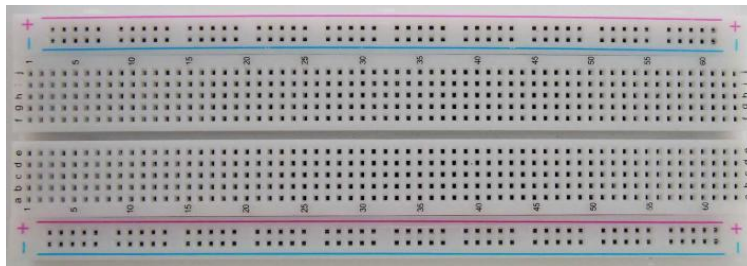
UNO R3 Controller Board: The **Arduino Board** is like the **brain** of your project. It's a small computer that you can program to control the other parts. You write instructions (code) on a regular computer and upload them to the Arduino, and it follows those instructions to make things happen!

Key Function: It runs the program and sends signals to control things.



Breadboard: The **Breadboard** is a plastic board with lots of little holes. It lets you **easily connect** wires and components together without having to permanently solder (glue with metal) them. You just push the wires and component leads into the holes to build the circuit.

Key Function: A temporary, reusable way to connect components and wires.



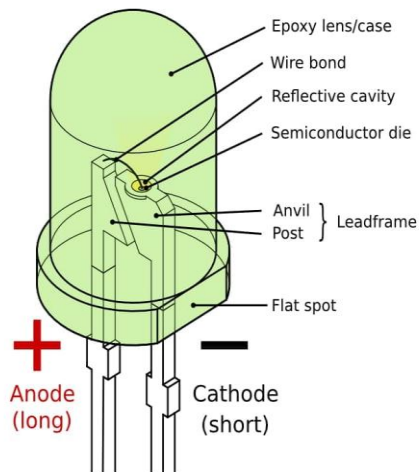
Jumper Wires: **Jumper Wires** are just simple wires used to **connect** all your components and the Arduino board to the breadboard. They complete the circuit so electricity can flow from the Arduino to the components and back again.

Key Function: To easily connect parts on the breadboard and to the Arduino's pins



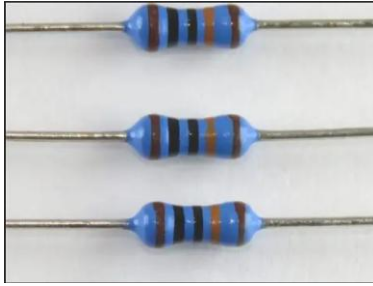
5mm LED: An **LED** is a tiny light bulb that lights up when you send electricity through it. It only allows electricity to flow in **one direction**, so you must connect the longer leg (the Anode or positive side) to the power source and the shorter leg (the Cathode or negative side) to the ground.

Key Function: Converts electrical energy into light.



100-ohm Resistor: A **Resistor** is a small component that **slows down** the flow of electricity (current). When you use an LED, you *must* use a resistor to protect the LED from getting too much electricity and burning out.

Key Function: Limits the amount of electrical current flowing in a circuit.



The value of resistance is known as an Ohm and its symbol is a Greek Omega symbol Ω . In this case, Digital Pin 10 is outputting 5v DC at (according to the Atmega datasheet) 40mA (milliamps), and your LEDs require (according to their datasheet) a voltage of 2v and a current of 35mA. Therefore, you need a resistor that will reduce the 5v to 2v and the current from 40mA to 35mA if you want to display the LED at its maximum brightness. If you want the LED to be dimmer, you could use a higher value of resistance.

Note NEVER use a value of resistor that is LOWER than needed. You will put too much current through the LED and damage it permanently. You could also damage other parts of your circuit.

The formula to work out what resistor you need is

$$R = (V_S - V_L) / I$$

where V_S is the supply voltage, V_L is the LED voltage, and I is the LED current. Your example LED has a voltage of 2v and a current of 35mA connected to a digital pin from an Arduino, which gives out 5 volts, so the resistor value needed would be

$$R = (5 - 2) / 0.035$$

which gives a value of 85.71.

Resistors come in standard values and the closest common value would be 100 Ω . Always choose the next standard value resistor that is HIGHER than the value needed. If you choose a lower value, too much current will flow through the resistor and will damage it.

How Do we Find a Certain Resistor?

A resistor is too small to contain easily readable labeling so resistors instead use a color code. Around the resistor you will typically find 4 colored bands; by using the color code in Table 2-1 you can find out the value of a resistor. Likewise, you can find the color code for a particular resistance.

Color	1st Band (Digit)	2nd Band (Digit)	3rd Band (Multiplier)	4th Band (Tolerance)
BLACK	0	0	10^0 (or x 1)	-
BROWN	1	1	10^1 (or x 10)	±1%
RED	2	2	10^2 (or x 100)	±2%
ORANGE	3	3	10^3 (or x 1,000)	-
YELLOW	4	4	10^4 (or x 10,000)	-
GREEN	5	5	10^5 (or x 100,000)	±0.5%
BLUE	6	6	10^6 (or x 1,000,000)	±0.25%
VIOLET	7	7	10^7	±0.1%
GREY	8	8	10^8	±0.05%
WHITE	9	9	10^9	-
GOLD	-	-	10^{-1}	±5%
SILVER	-	-	10^{-2}	±10%
NONE	-	-	-	±20%

How to Find a 100Ω Resistor?

To get a 100Ω resistor, you need the digits 1 and 0, followed by a multiplier that gives you the total value.

Band	Color	Value	Explanation
1st Band	BROWN	1	The first digit is 1.
2nd Band	BLACK	0	The second digit is 0. And The base value is now 10.
3rd Band	BROWN	10^1	The multiplier tells you how many zeros to add to the base value. $10 \times 10 = 100$.
4th Band	GOLD	±5%	This is the Tolerance , it means the resistor's actual value is very close to 100Ω.